

Abdul Wahid

EC-Council CHFI (Certified Hacking Forensic Investigator)

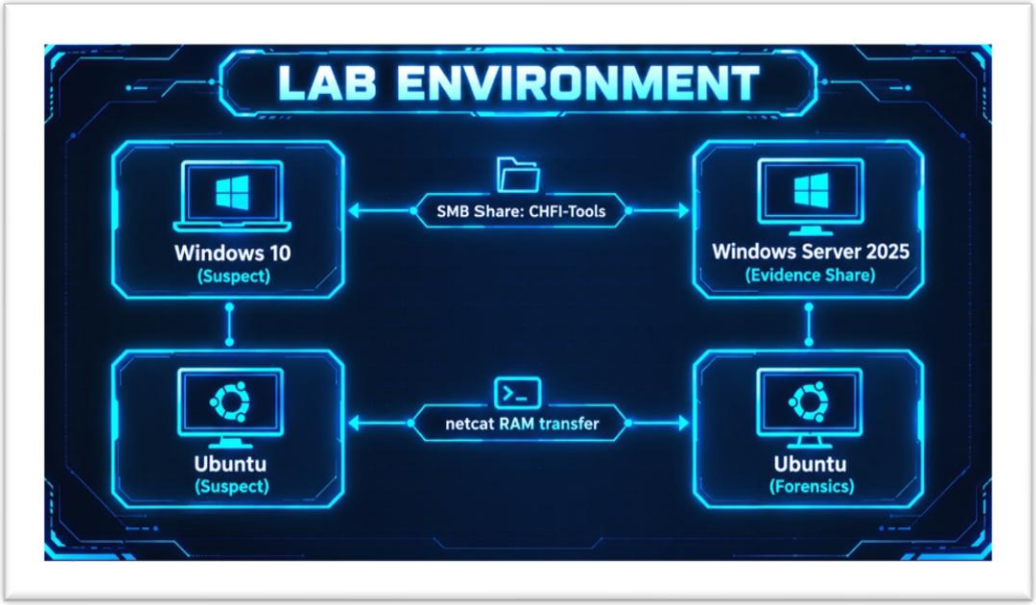
Data Acquisition & Duplication

EXECUTIVE SUMMARY

Background. This module documents seven practical labs on acquiring, converting, mounting and viewing digital evidence. Module 3 analysed images that were supplied. This module covers how such evidence is produced and prepared: a disk image from a Windows system, RAM captures from Windows and Linux systems, and read-only access to the resulting images on Windows and Linux forensic workstations. The work was carried out in a virtual training lab, and no real case data was involved.

Environment. The lab used four virtual machines. A Windows 10 machine acted as the suspect system. A Windows Server 2025 machine hosted the CHFI-Tools shared folder, which served as the external evidence drive. An Ubuntu Suspect machine was the Linux source system, and an Ubuntu Forensics workstation was used for conversion, mounting, analysis and as the receiver of the remote RAM dump.

Approach. Each lab followed the same cycle: prepare the source, acquire or convert, verify the output, then examine it read-only. Output was always written to a separate destination (the evidence share or a dedicated folder), never back onto the evidence being read. Volatile data was captured before any other work on the suspect systems, in line with the order of volatility.



Principal findings.

1. **Disk imaging (Lab 1).** A secondary virtual disk was added to the Windows 10 machine, initialised, partitioned and formatted as NTFS, and filled with test data. A bit-by-bit image of the physical drive was then taken with dd and saved to the evidence share as Windows_Evidence_001.dd. An MD5 hash of the image was generated to fingerprint it. The end-of-run "drive cannot find the sector requested" message was expected for this procedure.

2. **Format conversion (Lab 2).** The EnCase image Windows_Evidence_001.E01 was converted into a raw dd image on the Ubuntu Forensics workstation using xmount. This produced a Windows_Evidence_001.dd file and a matching .info file, and the original E01 was left unmodified.

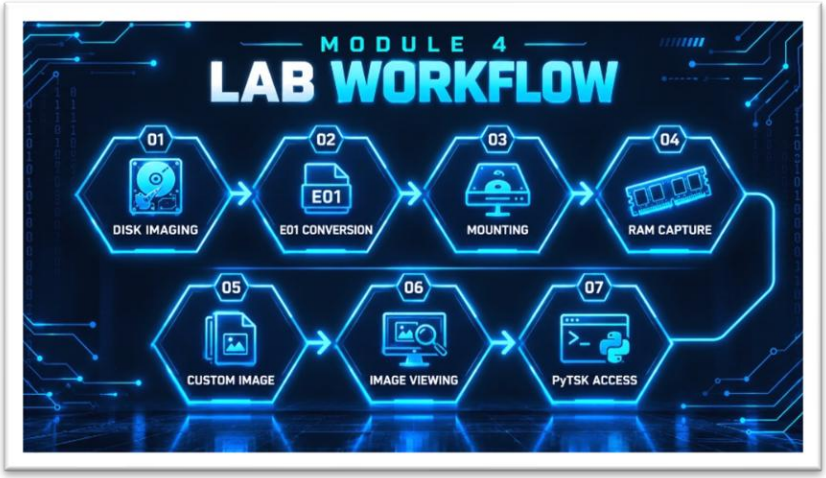
3. **Mounting images (Lab 3).** The converted dd image was mounted read-only with the mount command, and its folders (Audio Files, images, Other Files, Outlook Files, Songs, text and the system folders) were listed. A separate Mac HFS+ image, MAC_Evidence_001.dd, was attached through a loop device and opened, and its hidden system items (.fseventsd, .Spotlight-V100, .Trashes, .DS_Store and the journal files) were made visible.

4. **RAM acquisition (Lab 4).** Memory was captured from the Windows 10 machine with Belkasoft RAM Capturer, which produced a .mem dump. On the Ubuntu Suspect machine, memory was captured locally with fmem and dd, and with the LiME kernel module in lime format. A third capture was sent over the network from the Ubuntu Suspect machine to the Ubuntu Forensics workstation with dd and netcat. The maintained versions of fmem and LiME were used instead of the older copies supplied with the tools.

5. **Customised imaging (Lab 5).** DiskExplorer for NTFS opened Windows_Evidence_002.dd, showed its partition table with two valid NTFS partitions, displayed sector data in hexadecimal, and exported a customised image copy (RDEM.img) to a separate folder.

6. ***Viewing an image (Lab 6).*** AccessData FTK Imager previewed Windows_Evidence_001.dd without altering it. The file tree, file contents, hexadecimal view, file properties and the Hex Value Interpreter were all used.

7. ***Scripted access (Lab 7).*** A Python script using pytsk3, the Sleuth Kit binding, opened Linux_Evidence_001.img inside a virtual environment, read its file system and listed the root directory, including lost+found, etc, home, var and the orphan-files entry.



Limitations.

- The Linux local RAM capture with dd may end with “No space left on device”. This produces a partial dump, which the lab accepts as a demonstration only.
- The lab procedure hashes only the Lab 1 image. The source disk was not hashed for comparison, and the RAM dumps were not hashed by the procedure.
- No hardware write blocker was used, because the sources were virtual disks.
- The remote RAM transfer sends data unencrypted over the network, which is acceptable in a closed lab but not in real casework.
- The dd, fmem and LiME steps depend on kernel version and installed kernel headers, so exact results can differ between systems.

Integrity. Images were treated as read-only wherever the tools allowed. The mount command used the read-only option, FTK Imager and DiskExplorer opened images without writing to them, and all outputs went to separate folders. Hash verification was limited to what the lab procedure produced, and this is stated openly in the Limitations above.

Contents

Create a dd Image of a System Drive 6

Verifying SMB on Server 2025 8

Add a secondary virtual hard disk to the Windows 10 VM 9

Create the Partition on the new secondary disk 10

Converting an E01 Evidence Image to Raw dd Format 15

Install xmount: 16

Convert and mount the E01 file: 16

Mount Images on a Linux Forensic Workstation 17

Mount the dd image with the mount command 18

Mount a Mac (HFS+) dd image using a loop device 19

Acquire RAM from Windows and Linux Workstations 21

Windows RAM capture (Belkasoft RAM Capturer) 22

Linux local RAM acquisition (Ubuntu Suspect) 23

Linux remote RAM acquisition (dd + netcat) 26

Create Customized Images from an Image Containing NTFS File System 28

Launch DiskExplorer for NTFS. 29

Verify that RDEM.img exists in the destination folder. 32

View Contents of Forensic Image File 33

Access a Disk Image Using PyTSK Tool 38

Create a virtual environment and install pytsk3 inside it: 39

Create the Python script: 40

CONCLUSION 41

Create a dd Image of a System Drive

Adapted procedure – secondary disk method with full shared folder + partition steps

Lab Setup

Virtual machines needed

- Windows 10 VM (the suspect machine you'll image)
- Windows Server 2025 VM (hosts the shared folder CHFI-Tools, which acts as your "external drive")

Other requirements

- Internet connection and a web browser
- Administrator's privileges

Pre-checks

- Turn Real-Time Protection OFF on the Windows 10 VM (Windows Security → Virus & threat protection → Manage settings). Otherwise, it may block dd.exe.
- Make sure the Windows Server 2025 VM is running.

Lab concept: You first create a small secondary virtual disk inside the Windows 10 VM, initialize and partition it, create a shared folder on the Server, map it, then make a bit-by-bit physical image of the secondary disk and save it to the shared folder (treated as an external evidence drive).

Start the VMs

Turn on both Windows 10 and Windows Server 2025.

Create the Shared Folder on Windows Server 2025

Log in to the Windows Server 2025 VM.

Open File Explorer and go to a suitable location (e.g. C:\ or D:\).

Create a new folder and name it **CHFI-Tools**.

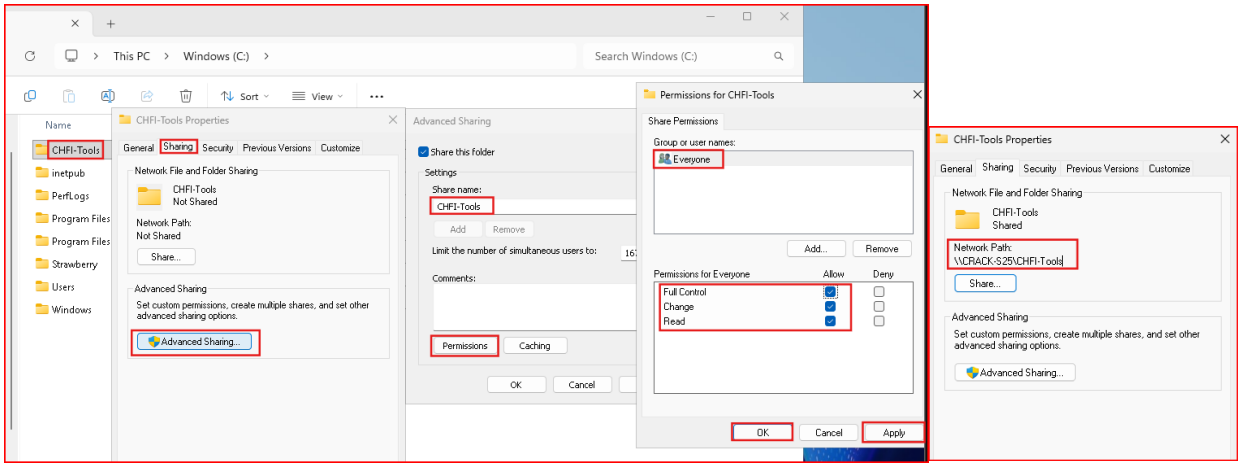
Right-click the **CHFI-Tools** folder → **Properties** → **Sharing** tab → **Advanced Sharing**.

Check **Share this folder**.

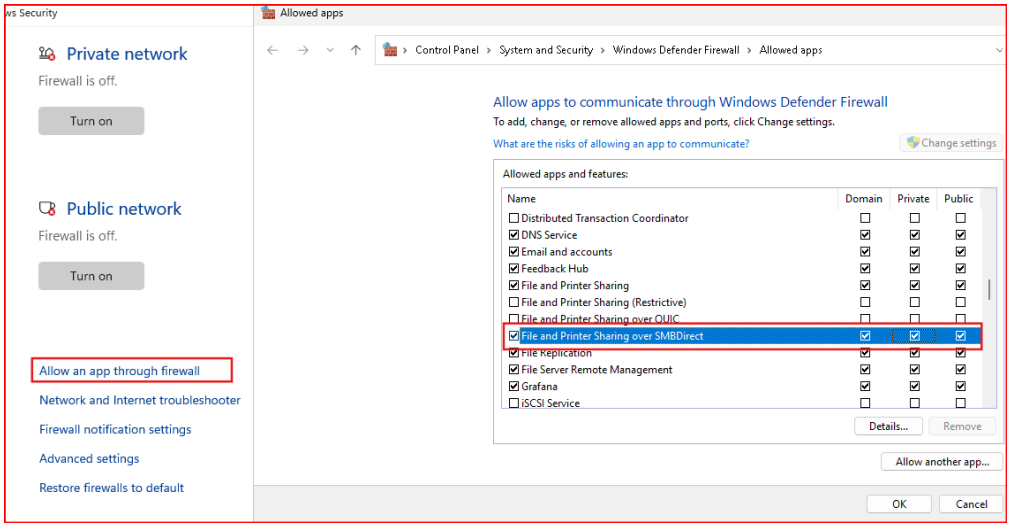
Click **Permissions** → Give **Full Control** to **Everyone** (or the specific account you will use).

Click **Apply** → **OK** → **OK**.

Note the network path (example: \\SERVER2025\CHFI-Tools). You will need this later.



Verifying SMB on Server 2025




```
PS C:\WINDOWS\system32> Get-WindowsOptionalFeature -Online -FeatureName SMB1Protocol


FeatureName      : SMB1Protocol
DisplayName      : SMB 1.0/CIFS File Sharing Support
Description      : Support for the SMB 1.0/CIFS file sharing protocol, and the Computer Browser protocol.
RestartRequired  : Possible
State            : Enabled
CustomProperties :
                   ServerComponent\Description : Support for the SMB 1.0/CIFS file sharing protocol, and the Computer
                   Browser protocol.
                   ServerComponent\DisplayName : SMB 1.0/CIFS File Sharing Support
                   ServerComponent\Id : 487
                   ServerComponent\Type : Feature
                   ServerComponent\UniqueName : FS-SMB1
                   ServerComponent\Deploys\Update\Name : SMB1Protocol
```

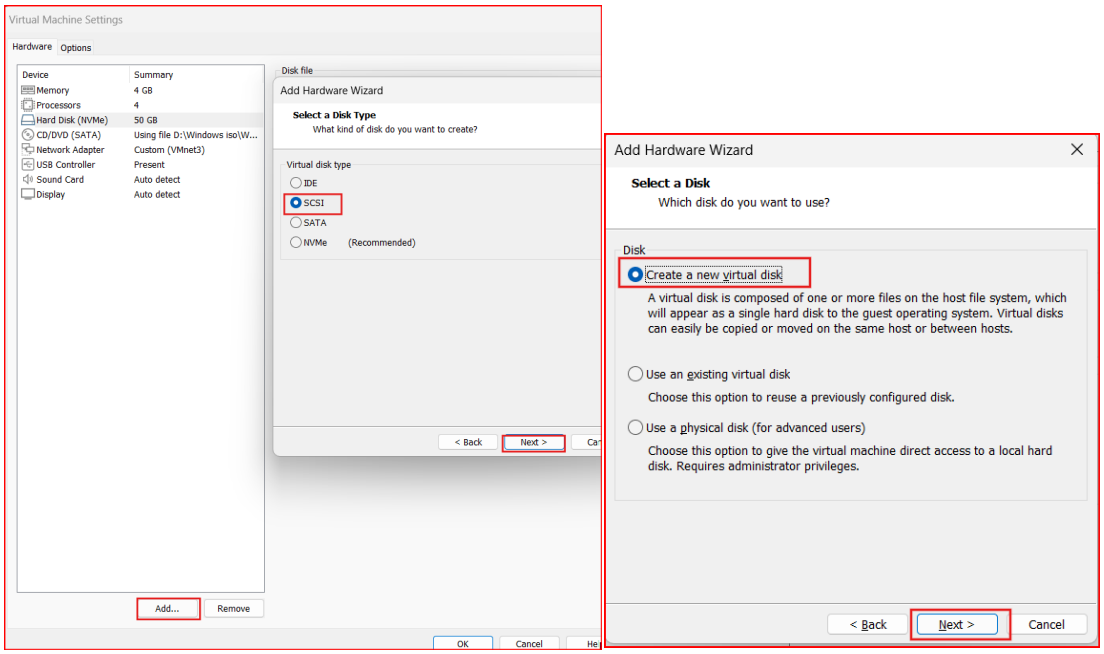
Add a secondary virtual hard disk to the Windows 10 VM

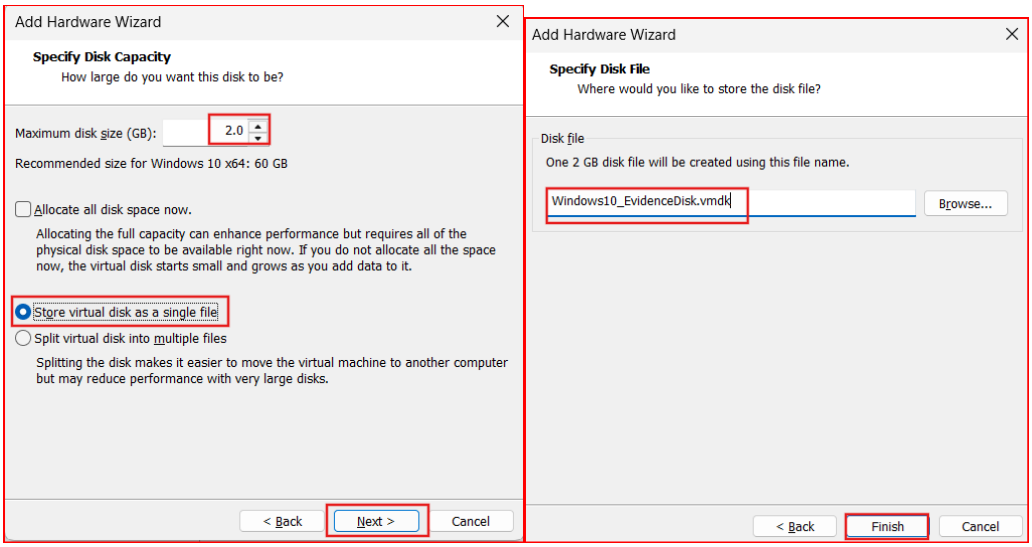
Shut down the Windows 10 VM if it is running.

In your hypervisor (VMware / VirtualBox / Hyper-V), edit the Windows 10 VM settings.

Add a new virtual hard disk (recommended size: 2 GB – 5 GB).

Start the Windows 10 VM again.





Create the Partition on the new secondary disk

Press **Windows key + X** → select **Disk Management** (or type diskmgmt.msc and press Enter).

You will see a new disk showing as **Unknown / Not Initialized** (usually Disk 1).

Right-click on the disk name (left side) → **Initialize Disk**.

Choose **GPT** (or MBR) → click **OK**.

Right-click on the **Unallocated** space → **New Simple Volume**.

Click **Next**.

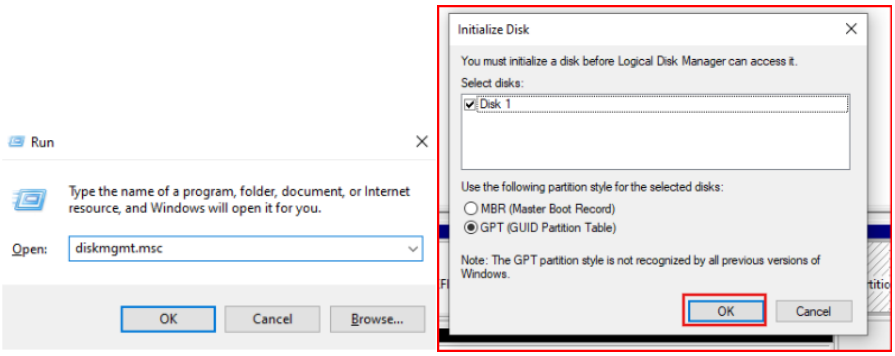
Leave the size as maximum → **Next**.

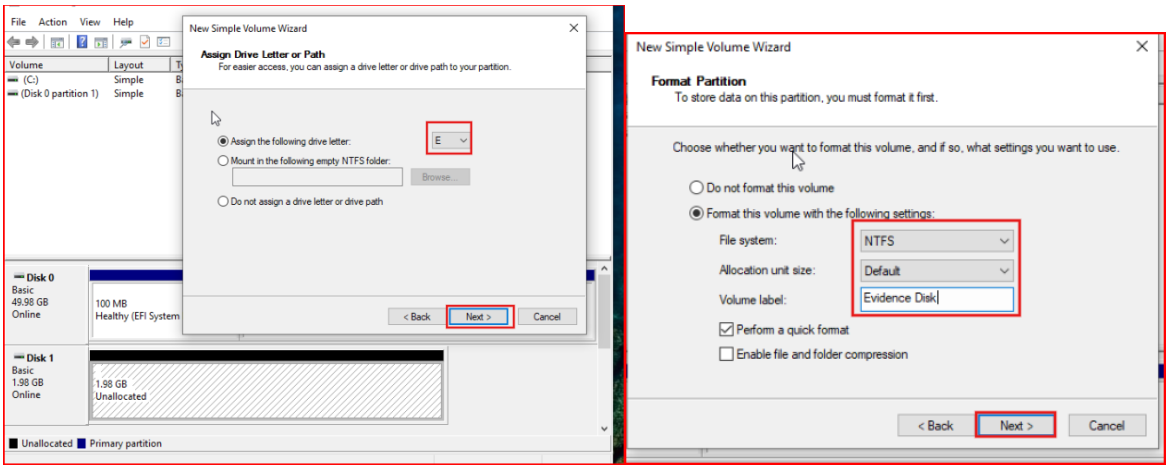
Assign a drive letter (e.g. **E:**) → **Next**.

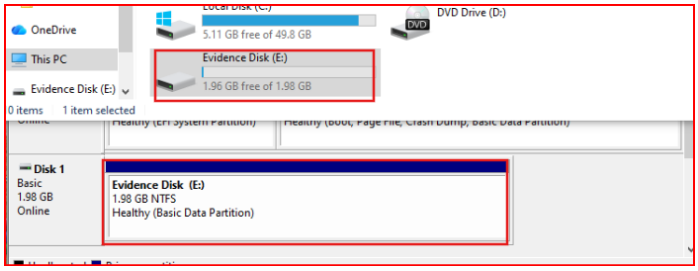
Format as **NTFS**, Allocation unit size = Default, Volume label = something like **EvidenceDisk** → check **Perform a quick format** → **Next** → **Finish**.

Wait until the volume shows as **Healthy**.

(Optional) Open the new drive (E:) and create a few test files/folders so the image contains meaningful data.







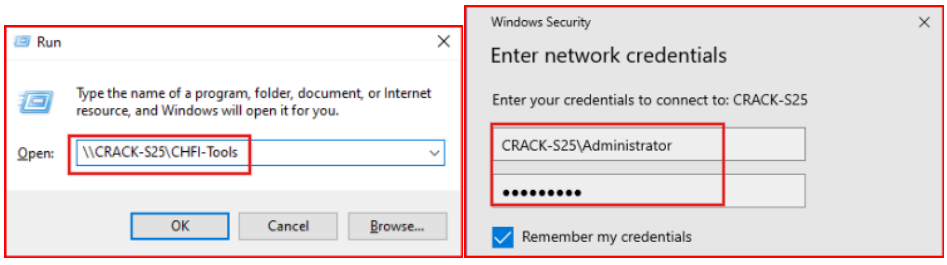
Connect to the Shared Folder from Windows 10

Open **File Explorer** on Windows 10.

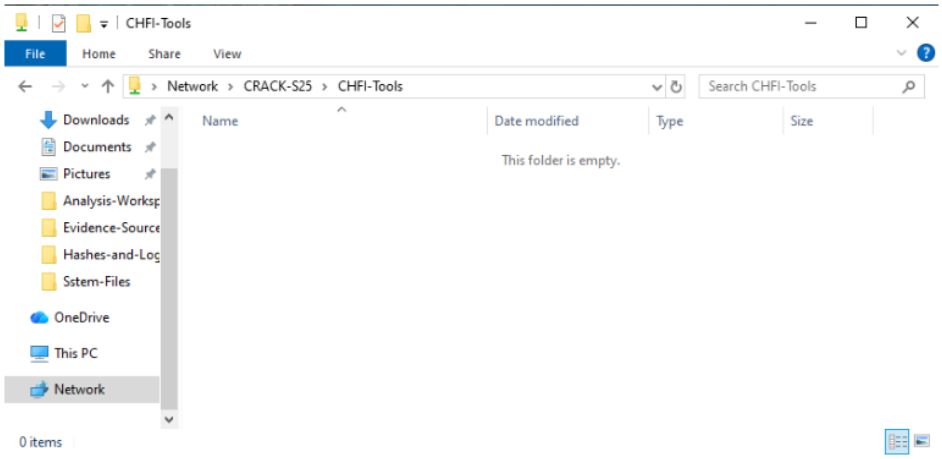
In the address bar, type the network path of the shared folder, for example:
\\SERVER2025\CHFI-Tools
(Use your actual Server name)

Press **Enter**.

If a login window appears, enter the Server credentials (username and password of the Server 2025 account that has access).



Once connected, you should see the contents of the **CHFI-Tools** folder.



Copy the dd tool

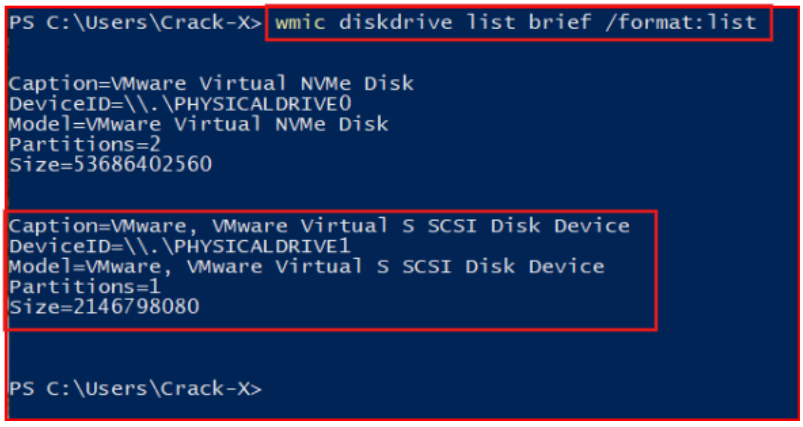
Copy the dd folder from the CHFI tool's location and paste it onto the Desktop of the Windows 10 VM.

Open PowerShell as Administrator

Type powershell in the search box → click **Run as administrator**. Click **Yes** if UAC appears.

List the drives

```
wmic diskdrive list brief /format:list
```



Note the secondary disk DeviceID (it will be \\.\PHYSICALDRIVE1 or higher — not PHYSICALDRIVE0).

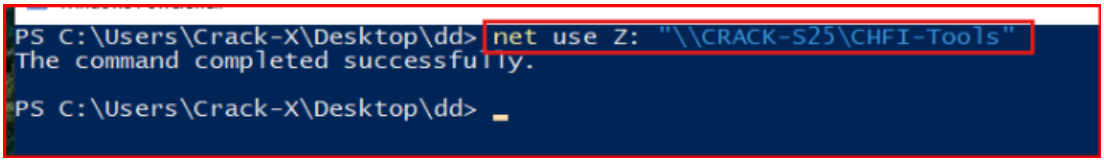
Go to the dd folder

```
cd C:\Users\Admin\Desktop\dd
```

(Adjust the path if your username is different.)

Map the shared folder as a drive letter

```
net use Z: \\SERVER2025\CHFI-Tools
```



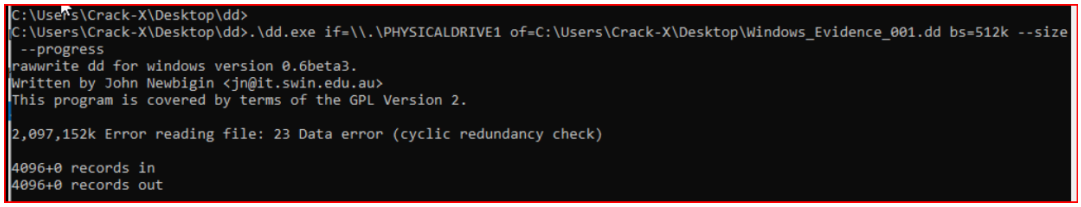
You should see "The command completed successfully."
(Replace with your actual server name and share path. Drive letter can be different.)

Create the Evidence folder

Open the mapped drive (Z:) in File Explorer and create a new folder named **Evidence**.

Run the acquisition

```
.\dd.exe if=\\.\PHYSICALDRIVE1 of=Z:\Evidence\Windows_Evidence_001.dd bs=512k
--size -progress
```

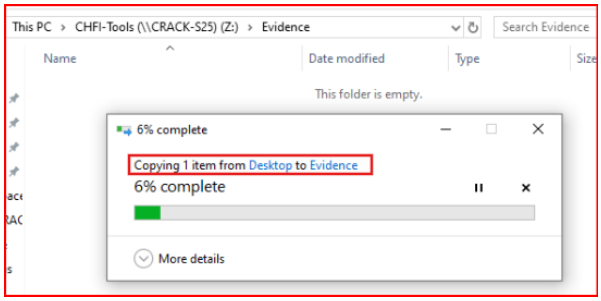


Replace PHYSICALDRIVE1 with the actual DeviceID of your secondary disk.

Use the correct drive letter if it is not Z:.

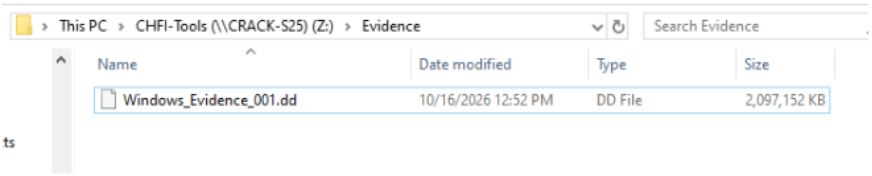
Wait for completion

This takes some time. When finished you will see records in and records out. An "Error reading file: 27 The drive cannot find the sector requested" message at the very end is normal.



Verify the image file

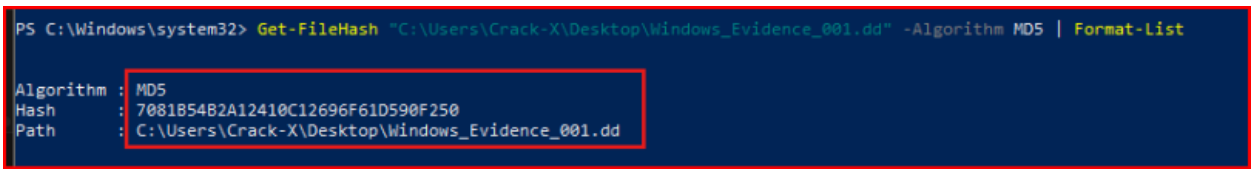
Open Z:\Evidence and confirm Windows_Evidence_001.dd exists (size will match the secondary disk you created).



Generate the MD5 hash

```
Get-FileHash 'Z:\Evidence\Windows_Evidence_001.dd' -Algorithm MD5 | Format-list
```

Note the Algorithm, Hash, and Path.



Clean up

- If you're short on space for later labs, delete the image file.
- Close all windows.
- Turn off the **Windows 10** VM.

Converting an E01 Evidence Image to Raw dd Format

Lab Setup

Virtual machines needed

- Ubuntu Forensics VM

Other requirements

- Internet connection
- Root privileges

Pre-checks

- Ensure the file Windows_Evidence_001.E01 is available in CHFI-Tools.

Lab concept: E01 images are not directly usable on Linux forensic workstations. Convert Windows_Evidence_001.E01 into a raw dd image using xmount.

Start the Ubuntu Forensics VM and log in.

Copy Windows_Evidence_001.E01 from CHFI-Tools to the Home directory (/home/).

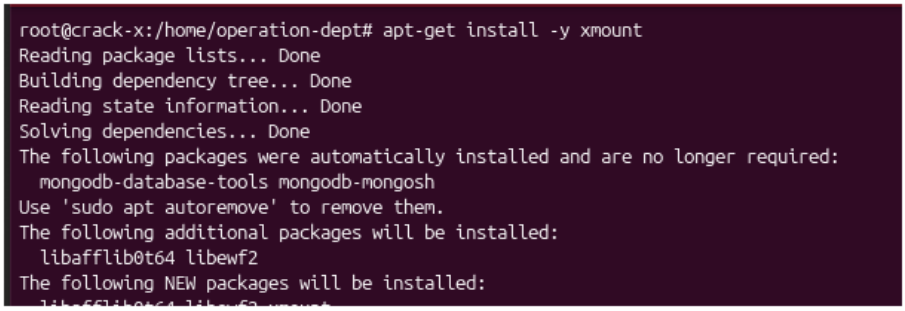
Open a terminal and switch to root:

```
sudo su
```



Install xmount:

```
apt-get install -y xmount
```



Convert and mount the E01 file:

```
xmount --in ewf Windows_Evidence_001.E01 /home/operation-dept/Documents
```

Verify the result in the Documents folder. You should see:

- Windows_Evidence_001.dd
- Windows_Evidence_001.info



Do not reboot or shut down the Ubuntu Forensics VM.

Lab 3 continues from this mounted image…

Mount Images on a Linux Forensic Workstation

Lab Setup

Virtual machines needed

- Ubuntu Forensics VM

Other requirements

- Root privileges

Pre-checks

- This lab continues from Lab 2. If the Ubuntu Forensics VM was not shut down, Windows_Evidence_001.dd is still mounted on /home/operation-dept/Documents.
- If you rebooted, the mount is gone and you must redo the xmount step.
- Windows_Evidence_001.E01 must still be in the Home directory.

Lab concept: You mount (1) the converted dd image using the mount command, and (2) a Mac (HFS+) dd image using a loop device.

Mount the dd image with the mount command

Open a terminal and switch to root:

```
sudo su
```

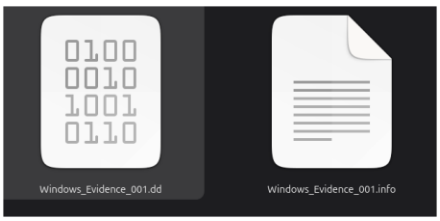
```
root@crack-x:/home/operation-dept#
```

Re-mount the E01 only if you rebooted (skip if the .dd file is still present):

```
xmount --in ewf Windows_Evidence_001.E01 /home/operation-dept/Documents
```

```
root@crack-x:/home/operation-dept# xmount --in ewf Windows_Evidence_001.E01 /home/operation-dept/Documents
root@crack-x:/home/operation-dept#
```

Confirm that Windows_Evidence_001.dd exists in the Documents folder.



Create a mount directory:

```
mkdir /mnt/dd
```

```
root@crack-x:/home/operation-dept# mkdir /mnt/dd
```

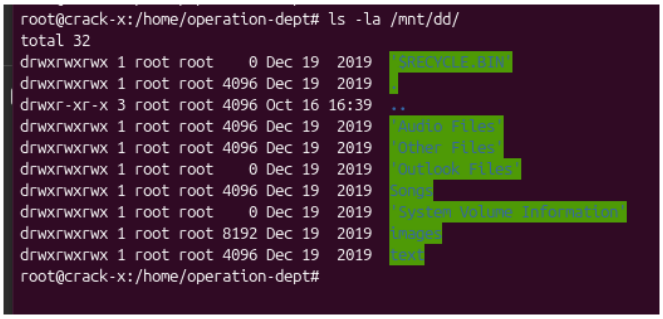
Mount the image read-only:

```
mount -o ro /home/operation-dept/Documents/Windows_Evidence_001.dd /mnt/dd/
```

```
root@crack-x:/home/operation-dept# mount -o ro /home/operation-dept/Documents/Windows_Evidence_001.dd /mnt/dd/
root@crack-x:/home/operation-dept#
```

List the contents:

```
ls -la /mnt/dd/
```



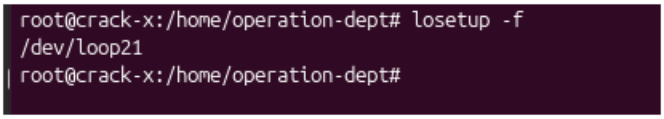
You should see folders such as $RECYCLE.BIN, Audio Files, images, Other Files, Outlook Files, Songs, System Volume Information, and text.

Mount a Mac (HFS+) dd image using a loop device

Copy MAC_Evidence_001.dd from CHFI-Tools to the Home directory (/home/operation-dept).

Find a free loop device:

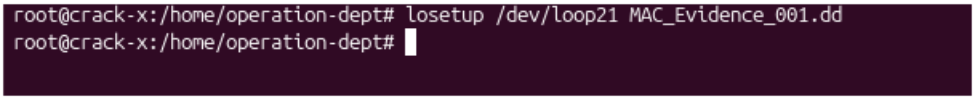
```
losetup -f
```



Note the output (example: /dev/loop21).

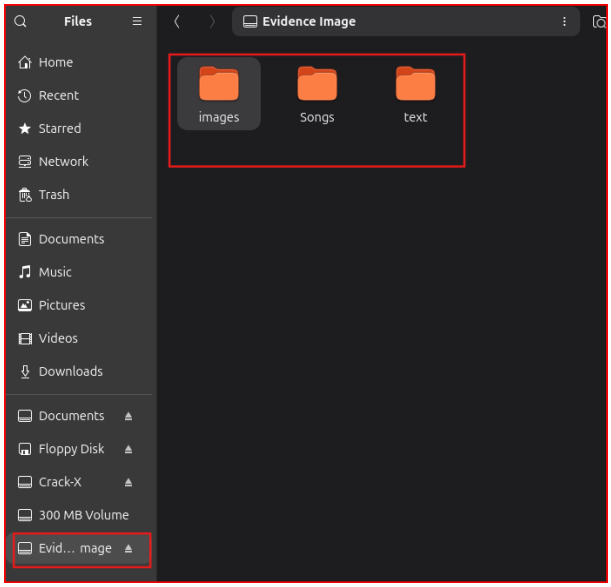
Attach the image to the loop device (run from Home directory):

```
losetup /dev/loop21 MAC_Evidence_001.dd
```

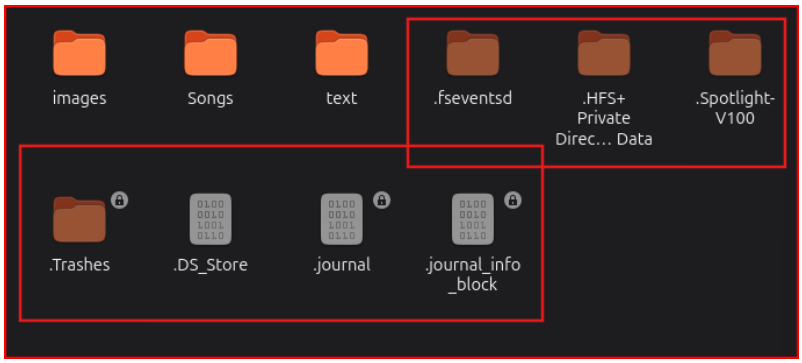


(Replace /dev/loop21 with the actual free loop device from the previous step.)

An Evidence Image icon appears. Open it to view the contents (images, Songs, text).



Press Ctrl+H to show hidden files such as .fseventsd, .HFS+ Private Directory Data, .Spotlight-V100, .Trashes, .DS_Store, .journal, and .journal_info_block.



Close all open windows…

Acquire RAM from Windows and Linux Workstations

Lab Setup

Virtual machines needed

- Windows 10 VM (suspect Windows machine)
- Ubuntu Suspect VM (suspect Linux machine)
- Ubuntu Forensics VM (receives the remote RAM dump)

Other requirements

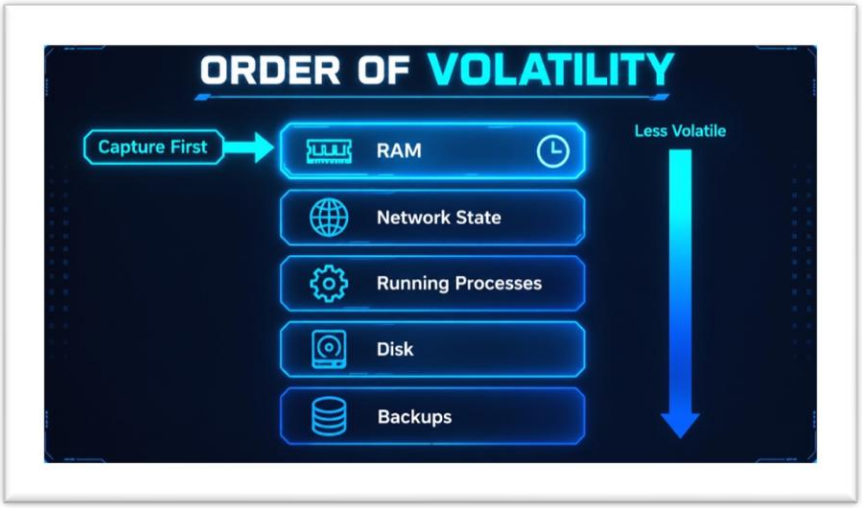
- Internet connection (for installing packages)
- Administrator/root privileges

Pre-checks

- Turn Real-Time Protection OFF on Windows 10.
- Confirm the IP address of Ubuntu Forensics with ip a (needed in Part C).

Lab concept: RAM holds volatile data that is lost when power is lost, so you capture it first. You will use Belkasoft RAM Capturer on Windows, and fmem, LiME, dd and netcat on Linux (local and remote acquisition).

Windows RAM capture (Belkasoft RAM Capturer)



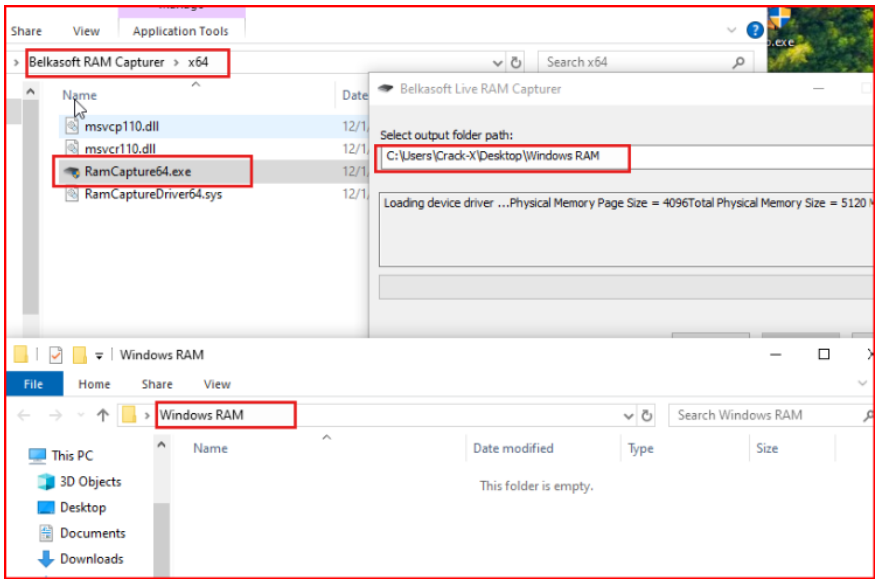
Start Windows 10 and Ubuntu Suspect.

Log in to Windows 10

Copy the **Belkasoft RAM Capturer** folder from CHFI-Tools to the Desktop.

Go to Desktop → Belkasoft RAM Capturer → RamCapturer → x64 and run RamCapture64.exe.

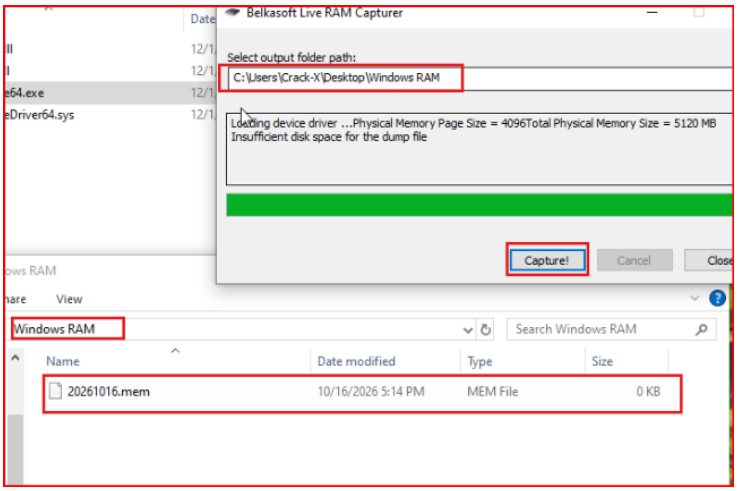
Create a folder named **Windows RAM** on the Desktop (or any local drive with sufficient space).



In the capturer window, set the output folder path to the **Windows RAM** folder and click **Capture.**

Wait for the progress bar to finish, then click **Close**.

Confirm that a dump file in yyyymmdd.mem format exists in the Windows RAM folder.

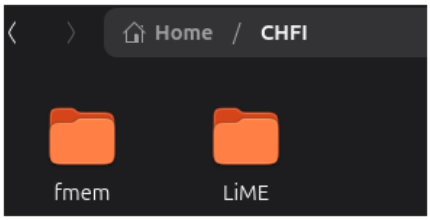


Close all windows……

Linux local RAM acquisition (Ubuntu Suspect)

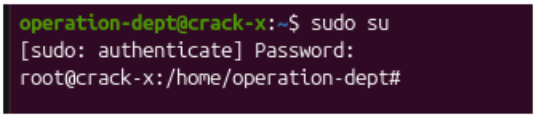
Switch to Ubuntu Suspect and log in as james with password toor.

Copy the **fmem** and **LiME** folders from CHFI-Tools into the Home directory.



Open a terminal and switch to root:

```
sudo su
```



Install and load fmem:

Using the updated (maintained) version of fmem instead of the old one from CHFI.

```
cd /home/operation-dept

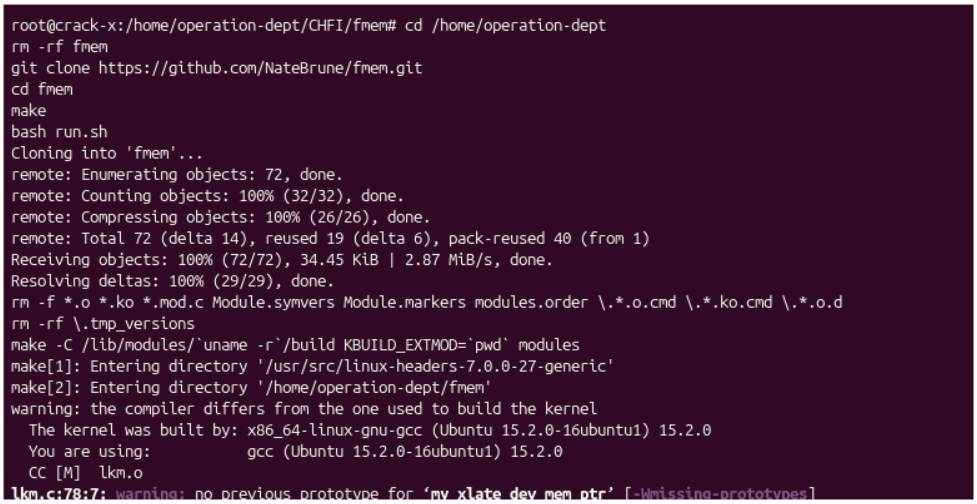
rm -rf fmem

git clone https://github.com/NateBrune/fmem.git

cd fmem

make

bash run.sh
```





Note: **Ensure you have your kernel headers should not be missing**

If Missing (Optional)

```
root@crack-x:/home/operation-dept/CHFI/fmem# apt update
apt install -y linux-headers-$(uname -r)
Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:2 https://repo.mongodb.org/apt/ubuntu focal/mongodb-org/4.4 InRelease [4,009 B]
Err:2 https://repo.mongodb.org/apt/ubuntu focal/mongodb-org/4.4 InRelease
  The following signatures couldn't be verified because the public key is not available: NO_PUBKEY 656408E390CFB1F5
Hit:3 http://pk.archive.ubuntu.com/ubuntu resolute InRelease
Get:4 http://pk.archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:5 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [602 kB]
```

Acquire RAM locally with dd:

```
dd if=/dev/fmem of=/home/operation-dept/ubuntu_local_ram.dd bs=1MB
```

```
root@crack-x:/home/operation-dept/fmem# dd if=/dev/fmem of=/home/operation-dept/ubuntu_local_ram.dd bs=1MB
```

This may take several minutes and can end with “No space left on device” (acceptable for the lab).

Verify ubuntu_local_ram.dd in Home, then permanently delete it to free space.



Install and run LiME:

Use the latest maintained version of LiME:

```
cd /home/operation-dept

rm -rf LiME

git clone https://github.com/504ensicsLabs/LiME.git

cd LiME/src

make
```

```
root@crack-x:/home/operation-dept/LiME/src# cd /home/operation-dept
rm -rf LiME
git clone https://github.com/504ensicsLabs/LiME.git
cd LiME/src
make
Cloning into 'LiME'...
```

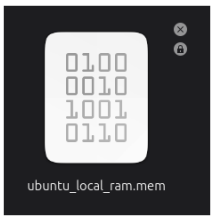
```
  LD [M]  lime.ko
  BTF [M] lime.ko
Skipping BTF generation for lime.ko due to unavailability of vmlinux
make[2]: Leaving directory '/home/operation-dept/LiME/src'
make[1]: Leaving directory '/usr/src/linux-headers-7.0.0-27-generic'
strip --strip-unneeded lime.ko
mv lime.ko lime-7.0.0-27-generic.ko
root@crack-x:/home/operation-dept/LiME/src#
```

```
insmod lime-7.0.0-27-generic.ko "path=/home/operation-
dept/ubuntu_local_ram.mem format=lime"
```

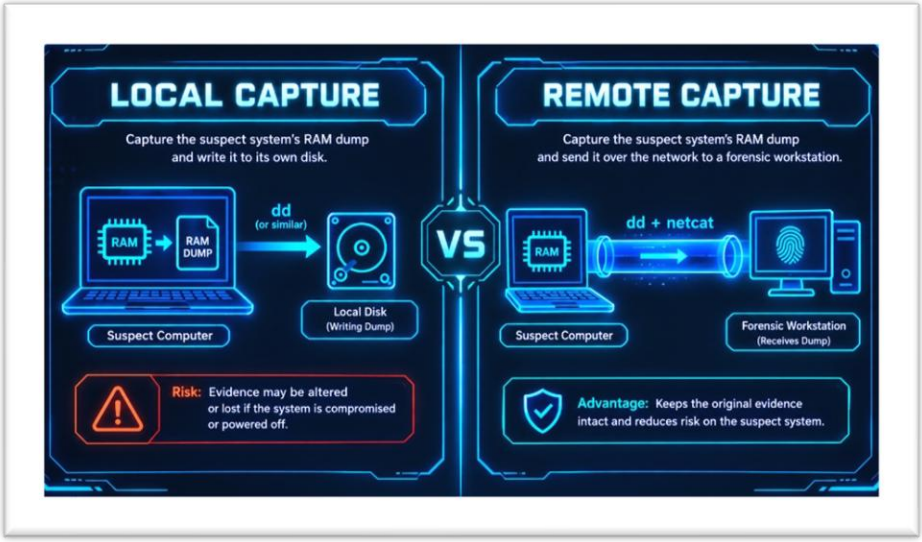
```
root@crack-x:/home/operation-dept/LiME/src# insmod ./lime-7.0.0-27-generic.ko path=/home/operation-dept/ubuntu_local_ram.mem format=lime
```

Use the exact .ko filename produced by make.

Verify that ubuntu_local_ram.mem appears in the Home directory.



Linux remote RAM acquisition (dd + netcat)



Switch to Ubuntu Forensics, open a terminal, and start the listener:

```
sudo su
```

```
nc -l 1234 > /home/operation-dept/ubuntu_remote_ram.dd
```

```
operation-dept@crack-x:~$ sudo su
[sudo: authenticate] Password:
root@crack-x:/home/operation-dept#
root@crack-x:/home/operation-dept# nc -l -p 1234 > /home/operation-dept/ubuntu_remote_ram.dd
```

Leave this terminal running…

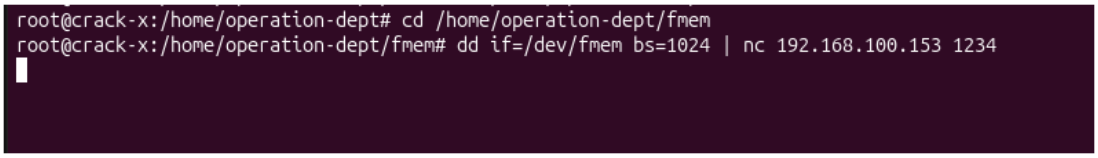
Switch to Ubuntu Suspect, open a new root terminal, and send the RAM dump:

```
sudo su

cd /home/operation-dept/fmem

bash run.sh          # only if /dev/fmem is not already loaded

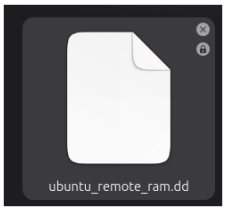
dd if=/dev/fmem bs=1024 | nc <Ubuntu-Forensics-IP> 1234
```



Replace <Ubuntu-Forensics-IP> with the actual IP of Ubuntu Forensics.

Allow the transfer to run for several minutes, then press Ctrl+C to stop.

On Ubuntu Forensics, confirm that ubuntu_remote_ram.dd exists in /home/operation-dept.



Close all windows on Ubuntu Forensics and Ubuntu Suspect...

Turn off Windows 10, Ubuntu Forensics, and Ubuntu Suspect…

Create Customized Images from an Image Containing NTFS File System

Lab Setup

Virtual machines needed

- Windows Server 2025 VM

Other requirements

- Administrator's privileges
- About 25 GB of free disk space

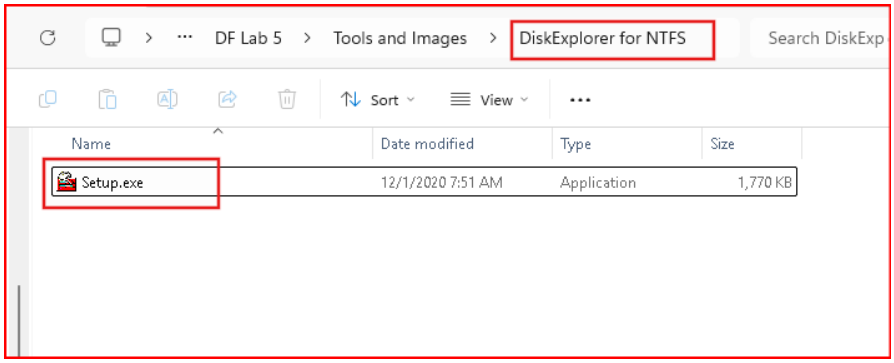
Pre-checks

- Ensure Windows_Evidence_002.dd and **DiskExplorer for NTFS** are available in CHFI-Tools.

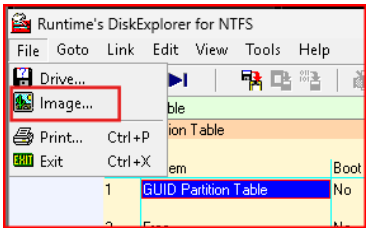
Lab concept: Use DiskExplorer for NTFS to open a raw forensic image (Windows_Evidence_002.dd), inspect its partition table and sectors in hex, and export a customized image copy to a file.

Log in to Windows Server 2025 as Administrator.

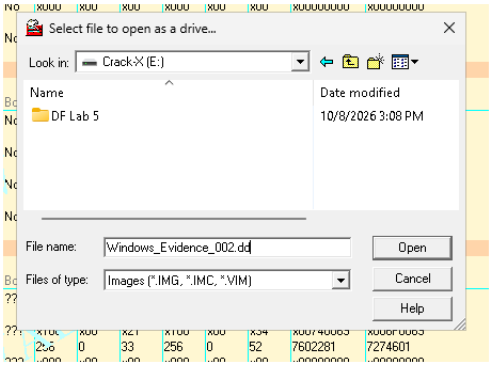
Launch DiskExplorer for NTFS.



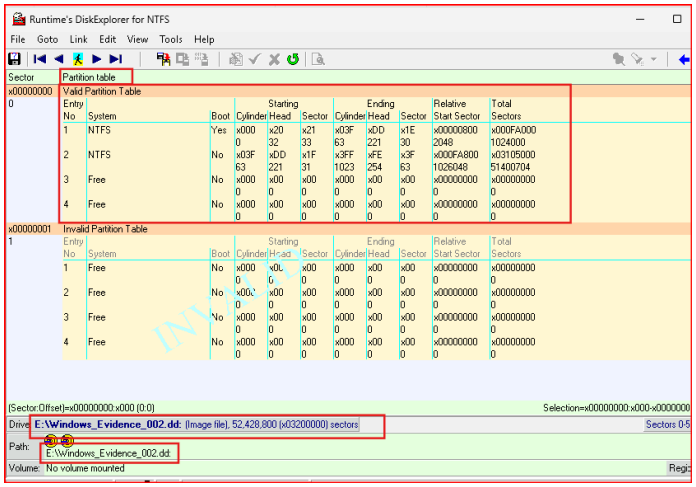
Go to **File → Image...**



Browse to the location of Windows_Evidence_002.dd in CHFI-Tools, select the file, and open it.

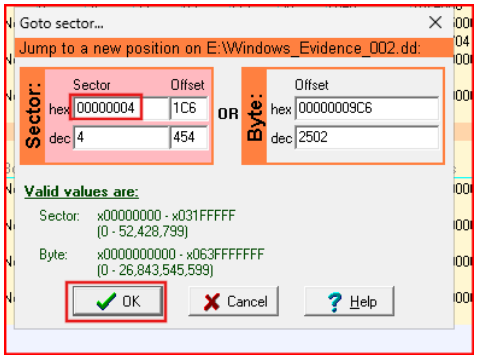


Review the partition table (you should see two valid NTFS partitions).

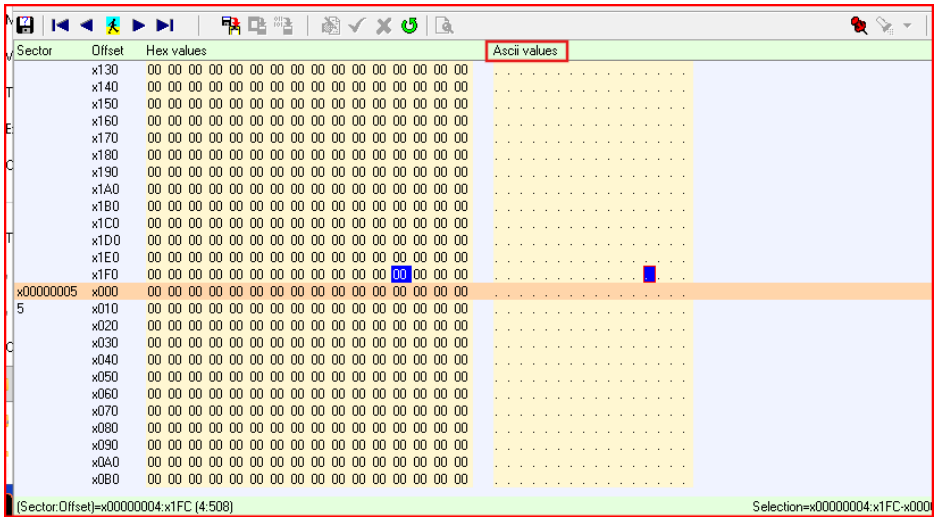


Go to **Goto → Sector...** (or press Ctrl+G).

Enter a sector value (example: 00000004) and click OK.

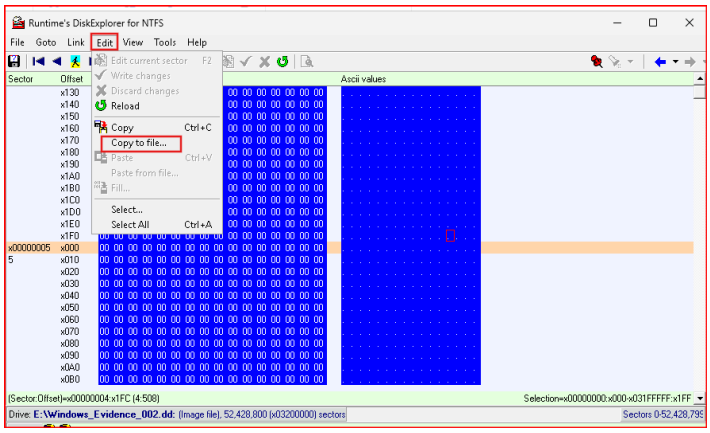


Go to **View → as Hex** (or press F3) to view the sector in hexadecimal and ASCII.

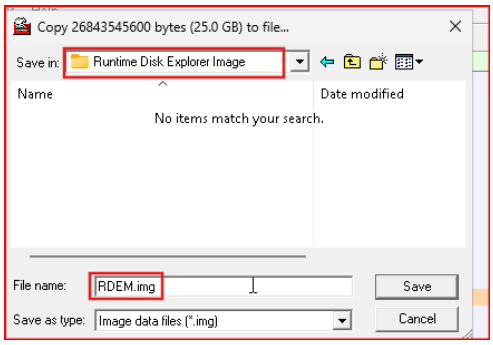


Go to **Edit → Select All** (Ctrl+A).

Go to **Edit → Copy to file...**

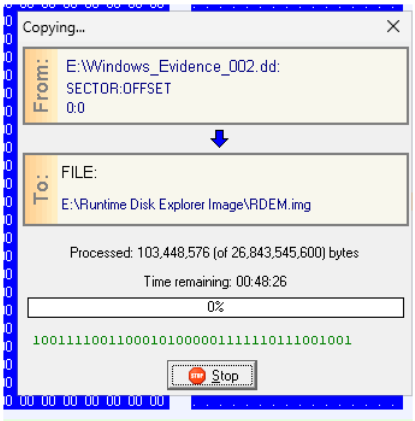


Create a folder named **Runtime Disk Explorer Image** (e.g. in Documents) and save the file as RDEM.img.

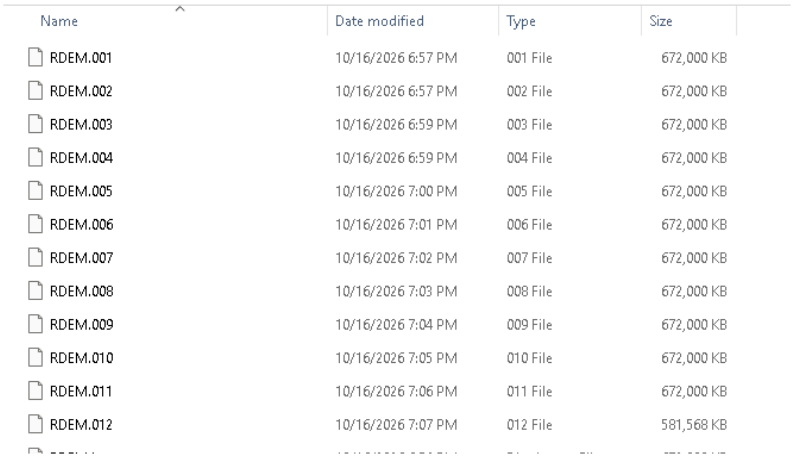


When asked "Do you want to split the image into several files?", click **No**.

Wait for the copy process to complete (approximately 5–10 minutes). The resulting file size will be around 25 GB.



Verify that RDEM.img exists in the destination folder.



Name	Date modified	Type	Size
RDEM.001	10/16/2026 6:57 PM	001 File	672,000 KB
RDEM.002	10/16/2026 6:57 PM	002 File	672,000 KB
RDEM.003	10/16/2026 6:59 PM	003 File	672,000 KB
RDEM.004	10/16/2026 6:59 PM	004 File	672,000 KB
RDEM.005	10/16/2026 7:00 PM	005 File	672,000 KB
RDEM.006	10/16/2026 7:01 PM	006 File	672,000 KB
RDEM.007	10/16/2026 7:02 PM	007 File	672,000 KB
RDEM.008	10/16/2026 7:03 PM	008 File	672,000 KB
RDEM.009	10/16/2026 7:04 PM	009 File	672,000 KB
RDEM.010	10/16/2026 7:05 PM	010 File	672,000 KB
RDEM.011	10/16/2026 7:06 PM	011 File	672,000 KB
RDEM.012	10/16/2026 7:07 PM	012 File	581,568 KB

(Optional) Delete RDEM.img after verification to free disk space.

Close all windows…

View Contents of Forensic Image File

Lab Setup

Virtual machines needed

- Windows Server 2025 VM

Other requirements

- Administrator's privileges

Pre-checks

- Ensure **AccessData FTK Imager** and the file Windows_Evidence_001.dd are available in CHFI-Tools.

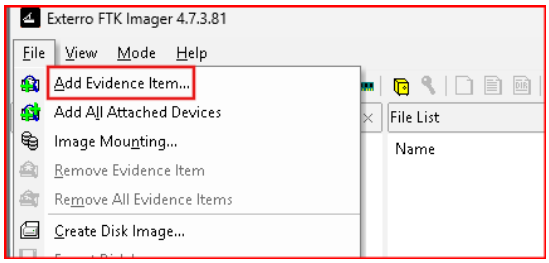
Lab concept: Use AccessData FTK Imager to preview a forensic image (Windows_Evidence_001.dd) without modifying the original evidence. Browse its file tree, view file contents, and inspect hex values and properties.

Log in to Windows Server 2025 as Administrator.

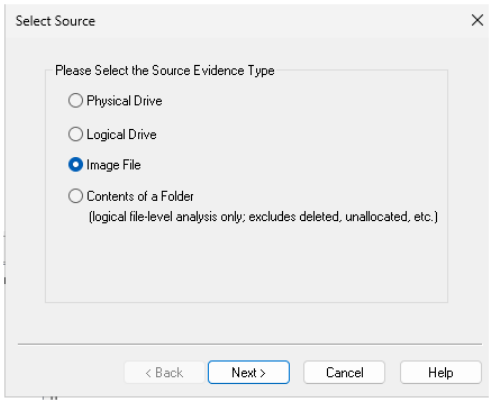
Navigate to CHFI-Tools -> **AccessData FTK Imager**.

Run AccessData_FTK_Imager_4.7.1.exe and complete the installation. Make sure **Launch AccessData FTK Imager** is checked at the end.

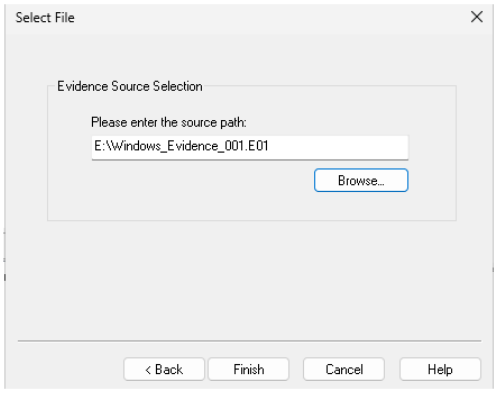
In FTK Imager, go to **File → Add Evidence Item...**



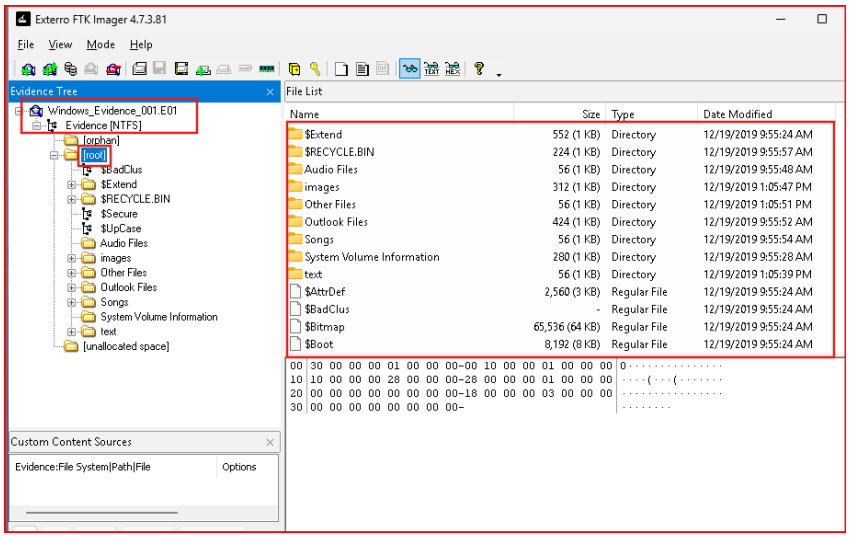
Select **Image File** and click **Next**.



Browse to Windows_Evidence_001.dd in CHFI-Tools and click **Finish**.

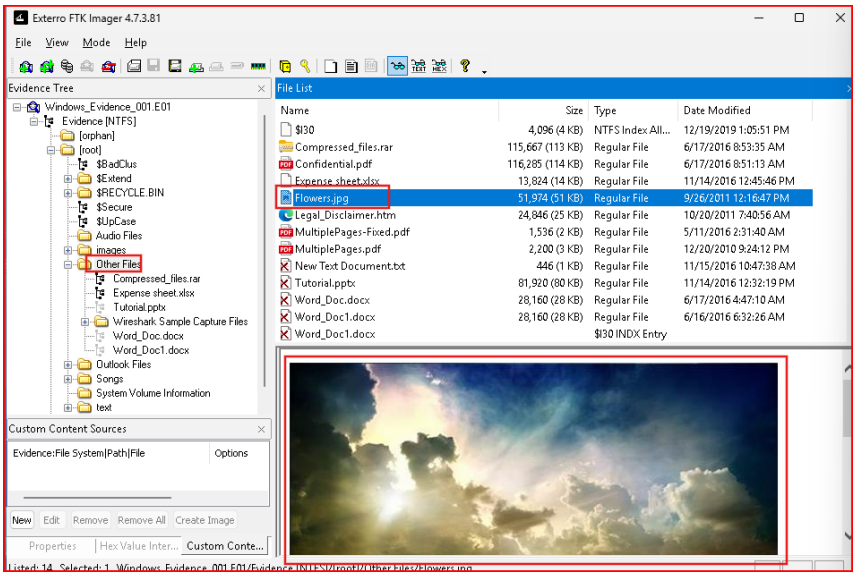


Expand the evidence tree:
Windows_Evidence_001.dd → Evidence [NTFS] → [root]
You should see folders such as $RECYCLE.BIN, Audio Files, images, Other Files, Outlook Files, Songs, System Volume Information, and text.

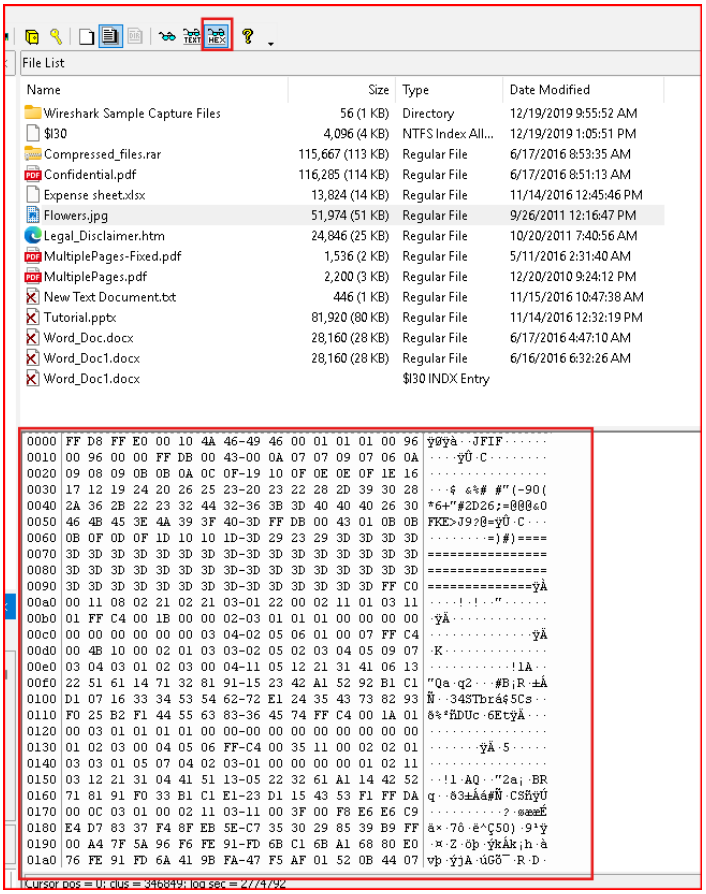


Select a folder (e.g. **Other Files**). Its files will appear in the File List.

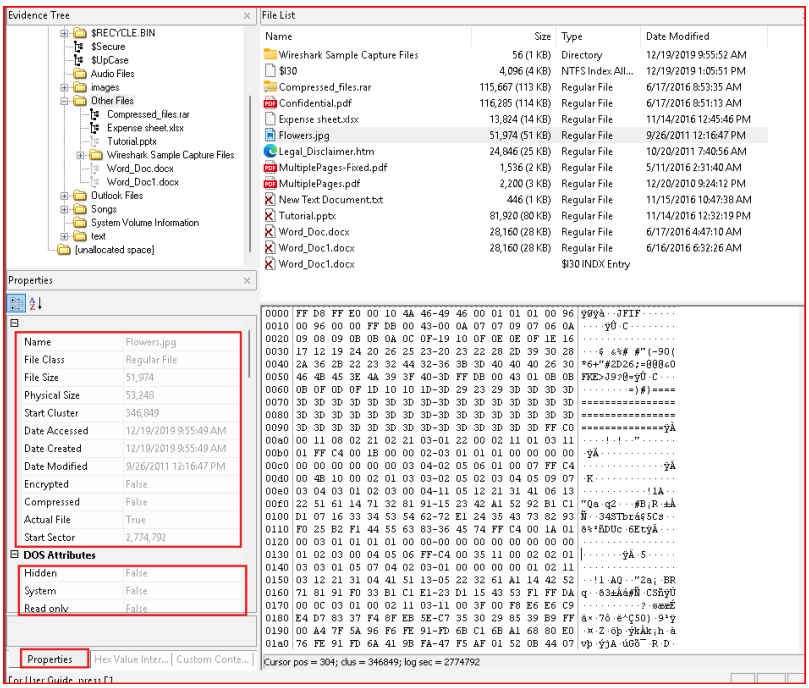
Click a file (e.g. Flowers.jpg). Its content preview appears in the bottom-right pane.



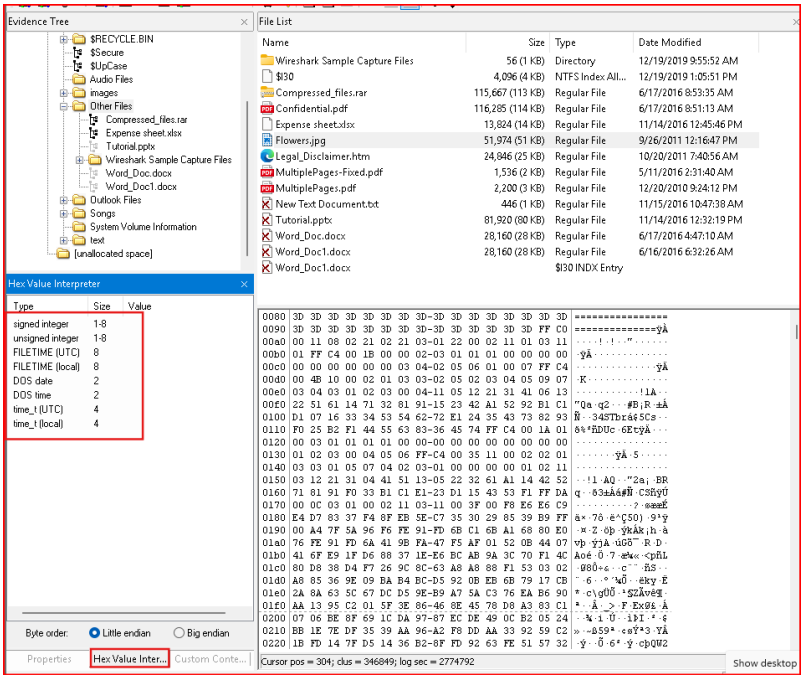
Click the **Hex** icon on the toolbar to view the hexadecimal values of the selected file.



Click the **Properties** tab in the lower-left pane to view file class, size, physical size, start cluster, and timestamps.



Click the **Hex Value Interpreter** tab to view interpreted values (signed/unsigned integer, FILETIME, DOS date/time, etc.).



Close all open windows…

Access a Disk Image Using PyTSK Tool

Lab Setup

Virtual machines needed

- Ubuntu Forensics VM

Other requirements

- Internet connection (needed to install pytsk3)
- Root privileges

Pre-checks

- Ensure the file Linux_Evidence_001.img is available in CHFI-Tools.
- Lab time: about 15 minutes.

Lab concept: PyTSK (pytsk3) is a Python binding for The Sleuth Kit. You write a short Python script that opens a disk image (Linux_Evidence_001.img), reads its file system, and lists the files and directories in the root.

Start the Ubuntu Forensics VM.

Copy Linux_Evidence_001.img from CHFI-Tools to the Home directory (/home/operation-dept).

Open a terminal and switch to root:

```
sudo su
```

Create a virtual environment and install pytsk3 inside it:

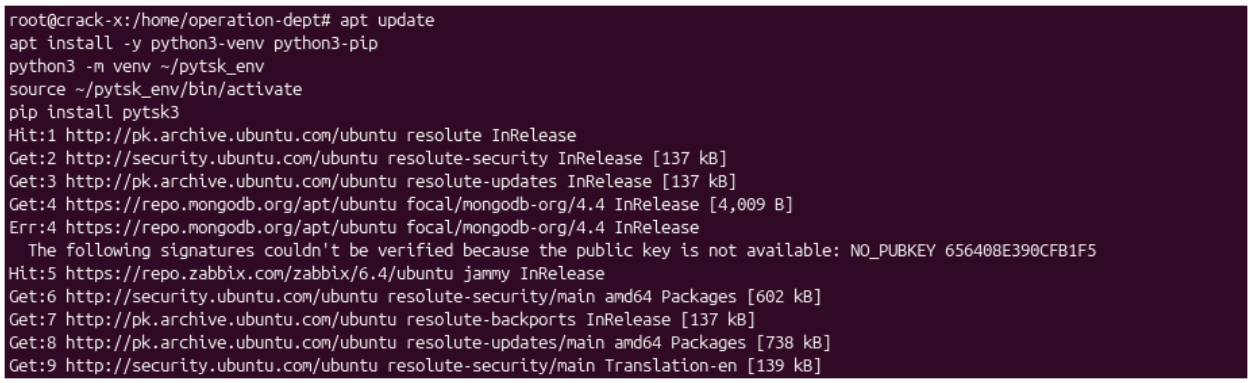
```
apt update

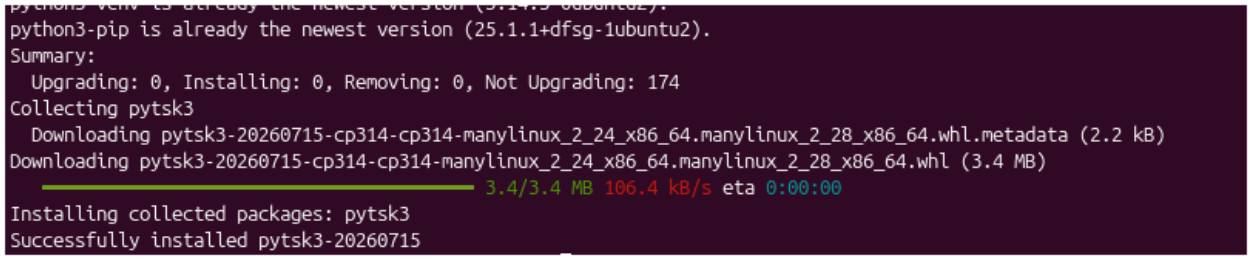
apt install -y python3-venv python3-pip

python3 -m venv ~/pytsk_env

source ~/pytsk_env/bin/activate

pip install pytsk3
```





Create the Python script:

```
gedit disk_analysis.py
```

Paste the following code exactly and save the file:

```
import pytsk3
# Open the disk image
disk_image = pytsk3.Img_Info('/home/operation-dept/Linux_Evidence_001.img')
# Open the file system
fs = pytsk3.FS_Info(disk_image)
# List the directories and files within the root directory
rootDir = fs.open_dir(path='/')
for entry in rootDir:
    print(entry.info.name.name.decode())
```

Run the script:

Before running check the evidence is at his right place or not




```
python3 ./disk_analysis.py
```

Review the output.
You should see the contents of the root directory, for example:
., .., lost+found, etc, media, bin, boot, dev, home, lib, lib64, mnt, opt, proc, root, run, sbin, snap, srv, sys, tmp, usr, var, vmlinuz, initrd.img, cdrom, .ismount-test-file, and $OrphanFiles.

```
(pytsk_env) root@crack-x:/home/operation-dept# python3 ./disk_analysis.py
.
..
lost+found
etc
media
bin
boot
dev
home
lib
lib64
mnt
opt
proc
root
run
sbin
snap
srv
sys
tmp
usr
var
vmlinuz
initrd.img
cdrom
.ismount-test-file
$OrphanFiles
(pytsk_env) root@crack-x:/home/operation-dept#
```

Close all open windows…

CONCLUSION

This module moved from analysing evidence to acquiring it. I created a bit-by-bit disk image with dd, converted an E01 image to raw format with xmount, mounted raw images read-only on Linux, and attached a Mac HFS+ image through a loop device. I captured volatile memory from Windows and Linux systems using four methods: Belkasoft RAM Capturer, fmem with dd, LiME, and dd piped over netcat. I then examined images with FTK Imager, DiskExplorer for NTFS and a pytsk3 script.

The main lessons are these. Volatile data must be captured first, because it is lost when power is lost. Every acquisition needs a fingerprint, and a hash only has value when it is compared against another one later. The acquisition method must suit the situation: local capture writes to the suspect machine and can overwrite data, while network capture avoids that but sends data unencrypted. Different formats such as raw dd and E01 need different handling, and evidence should always be opened read-only so the original is never changed.



The practical problems I met were Real-Time Protection blocking dd.exe, missing kernel headers when building fmem, the need to use the maintained versions of fmem and LiME, the expected dd error at the end of imaging, and limited disk space during large copies. I recorded them so the labs can be repeated.

These skills lead directly into later modules. The images produced and mounted here are the kind of evidence that file-system analysis, carving and timeline work depend on, as in Module 3, and the memory dumps are the starting point for volatile-data analysis. The limits recorded above, mainly the missing source-to-image hash comparison and the partial local RAM dump, are the items to improve in later work.